

Universidad de Antioquia
Facultad de Ingeniería
Arquitectura de Software

Informe de desarrollo Frontend Transeasy

Sara Mesa Gómez
Diego Jose Luis Botia

Introducción

En los últimos años, las entidades financieras han transformado sus procesos de transformación digital, migrando buena parte de sus servicios hacia plataformas web y móviles que permiten a los usuarios consultar información y realizar operaciones sin necesidad de acudir a una sucursal física. En este contexto, el desarrollo de interfaces web claras, funcionales y fáciles de usar, con los servicios del backend, se convierte en un componente clave de cualquier sistema bancario.

Se documentó el proceso de desarrollo del frontend del proyecto “Transacciones-Banco”, un sistema orientado a la gestión de clientes, cuentas y transacciones bancarias. Este trabajo partió de un backend base ya elaborado, implementado en Java con el framework Spring Boot, con servicios REST y utilizando una base de datos MySQL.

Este documento describe los objetivos del trabajo, las principales herramientas de software empleadas, la arquitectura propuesta para integrar ambas capas, el procedimiento seguido durante el desarrollo y las conclusiones obtenidas al finalizar el laboratorio.

Objetivo General

Desarrollar una interfaz web (frontend) en HTML, CSS y JavaScript que consuma los servicios REST del backend base (Spring Boot) del sistema de transacciones bancarias, permitiendo a los usuarios gestionar de manera intuitiva clientes, cuentas y transacciones.

Objetivos específicos

- Analizar la documentación y los endpoints REST expuestos por el backend base para identificar los recursos disponibles (clientes, cuentas y transacciones) y sus contratos de datos en formato JSON.
- Diseñar la estructura de páginas y componentes del frontend que cubra los flujos principales: consulta y registro de clientes, gestión de cuentas y registro o consulta de transacciones.
- Implementar un módulo de comunicación HTTP en JavaScript, basado en la API Fetch, que centralice las llamadas a los servicios REST y el manejo de respuestas y errores.
- Construir las vistas y la lógica de interacción de las operaciones CRUD sobre las entidades del dominio bancario.
- Verificar la correcta integración entre el frontend y el backend mediante pruebas manuales.
- Documentar las decisiones de arquitectura, las herramientas utilizadas y el procedimiento seguido durante el desarrollo del frontend.

Herramientas de software empleadas

A continuación se listan las principales herramientas y tecnologías utilizadas, tanto las que forman parte del backend base entregado (identificadas a partir del archivo pom.xml del proyecto) como las escogidas para el desarrollo del frontend.

Categoría	Herramienta
Lenguaje backend	Java 17
Framework backend	Spring Boot 3.5.11 (Web, Data JPA, Validation)
Gestor de dependencias	Maven (wrapper mvnw)
Base de datos	MySQL (mysql-connector-j)
Estructura frontend	HTML5
Estilos frontend	CSS3
Lógica frontend	JavaScript (ES6+)
Comunicación HTTP	Fetch API
Control de versiones	Git y GitHub
Entorno de desarrollo	Visual Studio Code

Arquitectura Propuesta

La arquitectura propuesta para el proyecto sigue un esquema cliente-servidor organizado en tres capas claramente separadas:

- Capa de presentación (frontend): páginas HTML/CSS/JavaScript que se ejecutan en el navegador del usuario y son responsables de la interfaz gráfica y de la interacción.
- Capa de aplicación / negocio (backend): controladores REST (@RestController), servicios con la lógica de negocio y DTO mapeados, implementados en Spring Boot.
- Capa de persistencia: repositorios de Spring Data JPA que gestionan el acceso a la base de datos relacional MySQL.

La comunicación entre el frontend y el backend se realiza mediante peticiones HTTP que intercambian datos en formato JSON, siguiendo el estilo arquitectónico REST.

Organización del Front

El frontend se organizó de forma modular, sirviendo como recursos estáticos del propio backend (carpeta src/main/resources/static de Spring Boot), lo que evita configuraciones adicionales de CORS al consumir la API desde el mismo origen:

- index.html – página de inicio y navegación general.
- clientes.html – listado y registro de clientes.
- cuentas.html – gestión de cuentas asociadas a un cliente.
- transacciones.html – registro y consulta de transacciones.
- css/styles.css – hoja de estilos común a todas las vistas.

- `js/app.js` – módulo central de comunicación HTTP.

Procedimiento

El desarrollo del frontend se llevó a cabo siguiendo los pasos que se describen a continuación:

1. Análisis del backend base: Se revisó el proyecto entregado (`pom.xml`, estructura de paquetes y controladores) para identificar el stack tecnológico (Spring Boot 3.5.11, Java 17), las entidades del dominio (cliente, cuenta y transacción) y los endpoints REST disponibles para cada una.
2. Definición del alcance del frontend: Se definieron las pantallas necesarias: gestión de clientes, gestión de cuentas y registro o consulta de transacciones, priorizando las operaciones de creación y consulta.
3. Definición de la estructura de carpetas: Se organizó el proyecto frontend en carpetas separadas para HTML, CSS y JavaScript, ubicando los archivos estáticos dentro de `src/main/resources/static` del backend.
4. Maquetado de las vistas (HTML/CSS): Se construyeron páginas con formularios y tablas para mostrar la información de clientes, cuentas y transacciones.
5. Implementación del módulo de comunicación con el API: Se creó el archivo `api.js`, que centraliza las llamadas HTTP mediante la función `fetch` y expone funciones reutilizables).
6. Conexión de las vistas con el backend: Cada página incorporó su propio script para invocar las funciones del módulo de comunicación, procesar las respuestas JSON y actualizar el DOM dinámicamente con los datos recibidos.
7. Validaciones y manejo de errores: Se agregaron validaciones básicas en los formularios del lado del cliente y manejo de errores en las peticiones.
8. Documentación: Finalmente, se documentó todo el proceso en el presente informe, incluyendo las herramientas empleadas, la arquitectura propuesta y las conclusiones del laboratorio.

Conclusiones

- La separación entre frontend y backend mediante una API REST facilitó el trabajo, permitiendo enfocarse en la interfaz sin modificar la lógica de negocio ya implementada en el backend base.
- El uso de JavaScript sin frameworks permitió comprender a fondo el funcionamiento de la API Fetch y el ciclo de vida de una petición HTTP.
- Servir el frontend desde los recursos estáticos del propio backend Spring Boot simplifica la integración al evitar configuraciones adicionales de CORS entre distintos orígenes.
- En general, el laboratorio permitió aplicar los conceptos de arquitectura cliente-servidor y de consumo de servicios REST en un caso de uso realista del dominio bancario.

Bibliografía

mesasg. (2026). Transacciones-Banco [Repositorio de código]. GitHub.

<https://github.com/mesasg/Transacciones-Banco>

Bass, L., Clements, P., & Kazman, R. (2021). Software Architecture in Practice (4a ed.). Addison-Wesley.